

# Audio-Conditioned Masked-LM (A-CMLM) ASR

Full training & inference pipeline — data, masking mathematics, architecture, decoding, validation

**A frame-synchronous, non-autoregressive ASR head that fills masked text conditioned on audio, and decodes it iteratively (OmniVoice-style).** This document explains every stage end to end: the data format and collator, the probability behind the masking, the addition-fusion architecture, the training forward/backward and loss, the iterative decoder, and how the model is validated. It is the A-CMLM extension of the frozen-HuBERT + self-attention filler-ASR head, keeping that recipe intact and adding a text stream.

A-CMLM ASR · FROZEN HUBERT + SA HEAD · TRAINING & INFERENCE PIPELINE

---

## 1. Overview: one idea in one line

The previous model predicted the whole transcript from audio **in a single shot** — a non-autoregressive frame classifier. A-CMLM generalises it in two ways:

1. **Text feedback** — the model also reads a partially-filled text hypothesis, with unknown slots marked `<mask>`.
2. **Iteration** — decoding runs in several passes; each pass commits a few slots, and later passes condition on those commits.

The previous model is exactly this new model run with **N=1 step from a 100%-masked start** (the all-`<mask>` text carries no information, so the forward pass reduces to "audio → text"). Everything below is how to train it so that **N > 1** iteration helps.

Because the labels are **frame-synchronous and positional** (character *i* is pinned to encoder frame *i*), two things come for free that speech-synthesis papers must engineer:

- **No duration model** — the canvas length `T` is the HuBERT frame count, read directly from the audio.
- **No cross-attention** — text position *i* and audio frame *i* share an index, so audio and text are fused by **addition** at each position.

## 2. Data format and the collator

### 2.1 Manifest

Input is a JSONL manifest, one utterance per line:

```
{"key": "103-1240-0000", "source": ".../wavs/103-1240-0000.wav", "target": "CHAPTER ONE MISSUS RACHEL  
LYNDE ..."}

```

Only `{audio_path, text}` is held in memory; audio is decoded lazily in the collator at batch time (no caching).

### 2.2 The frame-synchronous label (unchanged)

Per clip the audio is read (16 kHz mono), normalised by `Wav2Vec2FeatureExtractor`, and its encoder frame count is `T = compute_output_length(#samples)` ( $\approx 50$  fps). The target is built **left-packed** over those `T` frames:

```
y = [<s>] + char_ids + [</s>]*3 + [<fill>]*(T - len)

```

with the transcript lowercased, punctuation stripped, spaces → `|`, and `eos_repeat = 3`. The **duration budget** `n_keep = ceil(dur · 25)` splits the canvas into a **content region** (positions `0 ... n_keep-1`) and a fixed `<fill>` **tail** (`n_keep ... T-1`).

## 2.3 Three tokens that are all called "mask" — kept distinct

| token / mechanism                      | meaning                                                            | where                                                                 | when              |
|----------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------|-------------------|
| <code>mask_embed</code> (audio 20-30%) | replaces spans of <b>acoustic tap frames</b> with a learned vector | on the tap, before the add                                            | train only        |
| <code>&lt;mask&gt;</code> (id 33)      | hides a <b>text label slot</b>                                     | in <code>text_input_ids</code>                                        | train + inference |
| <code>&lt;pad&gt;</code> (id 29)       | a position that <b>doesn't exist</b> for a short clip in a batch   | <code>text_input_ids</code> , ignored via <code>attention_mask</code> | batched only      |

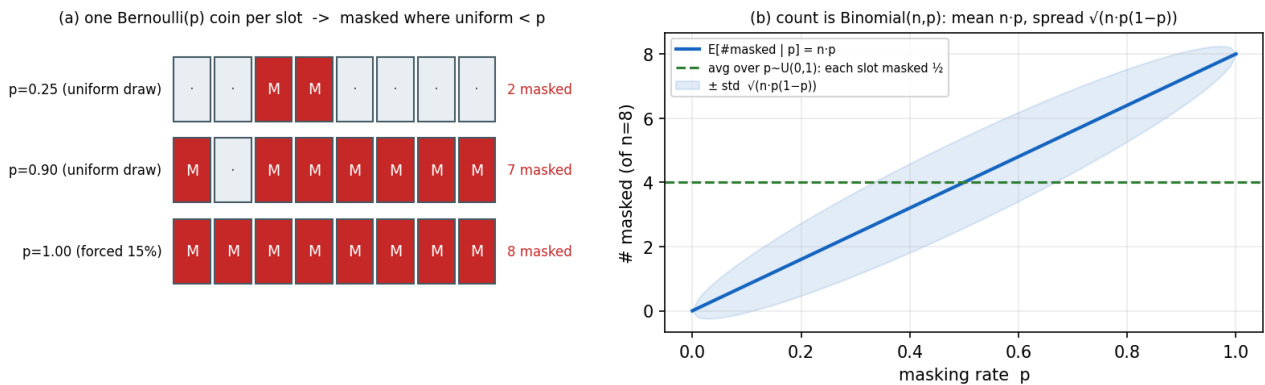
`mask_embed` corrupts audio; `<mask>` hides text; `<pad>` is batch filler. They are independent.

## 2.4 What the collator emits

Per batch: `input_values`, `attention_mask` (audio) + `text_input_ids` (the masked label) + `labels` (the true token at masked slots, `-100` elsewhere). Loss is therefore computed on the masked slots only (§5).

## 3. The masking mathematics

Masking is a **two-stage random process**, drawn fresh per clip. Understanding it explains both training and why the decoder starts the way it does.



### 3.1 Bernoulli( $p$ ): a biased coin

Each maskable slot is masked or not by a single yes/no trial. A **Bernoulli( $p$ )** variable is `1` (masked) with probability  $p$  and `0` otherwise:

$$P(X=1) = p, \quad P(X=0) = 1-p, \quad E[X] = p, \quad \text{Var}[X] = p(1-p)$$

The mean is  $p$  (bigger  $p \rightarrow$  more masking); the variance is largest at  $p=0.5$  and **zero at  $p=1$**  (all-masked is deterministic — the inference start).

### 3.2 Binomial( $n, p$ ): masking a whole sentence

With  $n$  maskable slots, each an independent Bernoulli( $p$ ), the **number masked**  $K$  is Binomial( $n, p$ ):

$$E[K] = n \cdot p, \quad \text{Var}[K] = n \cdot p \cdot (1-p), \quad P(K=k) = C(n, k) p^k (1-p)^{n-k}$$

So  $n=10$ ,  $p=0.3$  **expects** 3 masked, but any draw fluctuates around it (Figure (b)).

### 3.3 Why draw $p \sim \text{Uniform}(0,1)$ first (hierarchical randomness)

Rather than fixing  $p$ , each clip first draws its own rate  $p \sim U(0,1)$  (every value in  $[0,1]$  equally likely), then flips the per-slot coins. Averaging over the rate, the **marginal** probability a slot is masked is

$$P(\text{slot masked}) = \int_0^1 p \, dp = 1/2$$

— each slot is hidden half the time on average, but **how many neighbours share its fate swings clip-to-clip**, from "one letter missing" to "the whole sentence missing." This trains the model across every reconstruction difficulty, culminating in the fully-masked case the decoder begins from.

### 3.4 Oversampling the all-masked case

Because  $U(0,1)$  rarely lands near 1, yet the first decode step is 100% masked, the collator uses a **mixture**: with probability 0.15 it forces  $p = 1.0$ , else  $p \sim U(0,1)$ . This makes the decode-start regime well-trained rather than a rare tail.

### 3.5 In code (`DataCollatorACMLM`)

```
p = 1.0 if torch.rand(1).item() < 0.15 else torch.rand(1).item() # stage 1: the rate
region = torch.arange(1, n_keep) # maskable slots (exclude <s>=0)
sel = region[torch.rand(region.numel()) < p] # stage 2: Bernoulli(p) per slot
text_in[sel] = <mask> ; labels[sel] = y[sel] # hide input / grade truth
```

`torch.rand(...) < p` simulates a biased coin: a uniform value is below  $p$  exactly with probability  $p$ .

## 4. Architecture: addition fusion (no $y$ )

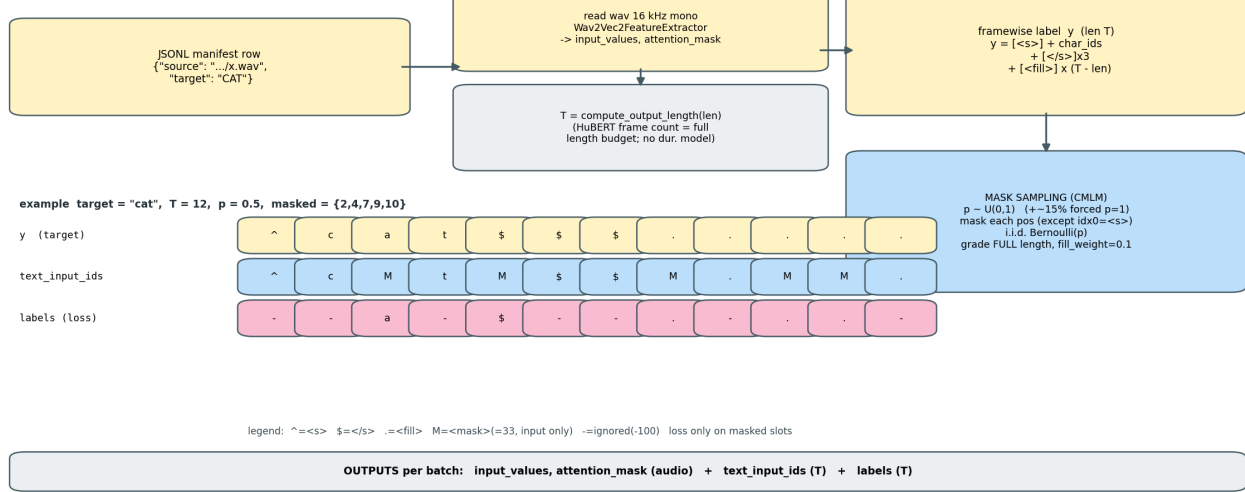
The vocabulary grows from 33 to **34** with a single input-only token `<mask> = 33` (never predicted; `lm_head` stays  $\rightarrow 33$ ). A small **text embedding** `E_text: Embedding(34, 1280)` is added into the existing stack:

```
h = tap (+ 20–30% audio mask, train-only) + PE + E_text(text_input_ids) → 8 SA → lm_head → 33
```

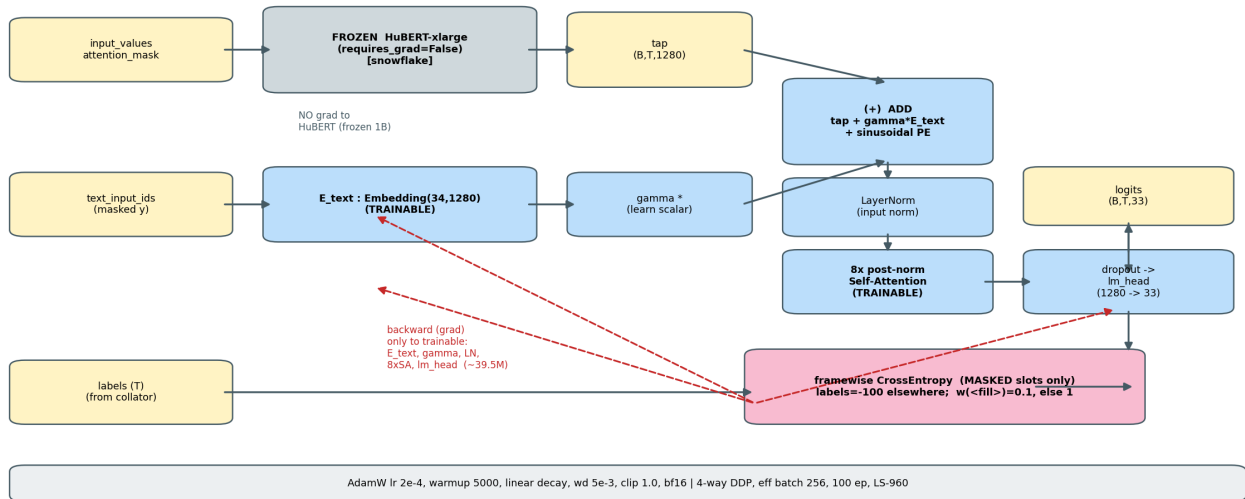
There is **no  $y$  scalar and no extra LayerNorm** — the learnable embedding already sets its own scale, and small initialisation (std 0.02) makes step 0 behave like the audio-only parent model, growing the text pathway during training. The frozen 1B-parameter HuBERT encoder is untouched; only the 8 self-attention layers, `lm_head`, `mask_embed`, and the tiny text embedding train ( $\sim 39.5\text{M}$ ).

## 5. Training: forward, loss, backward

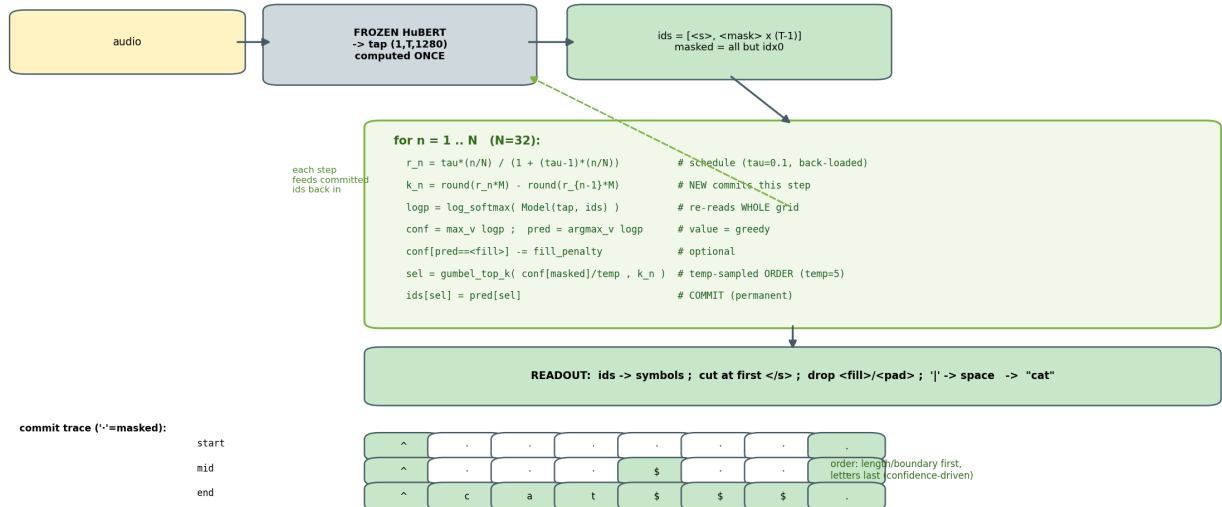
### A. COLLATOR + MASKING (per clip, at batch time — no caching)



### B. TRAINING (audio-conditioned masked-LM; forward -> loss -> backward)



### C. INFERENCE (OmniVoice iterative unmask; no duration model, no layer penalty)



## 5.1 Loss on masked slots only

The model sees the true text at unmasked positions, so grading them would just teach copying. The masked-LM objective grades **only the masked slots**, where the answer is hidden and must come from audio + context:

```
L = (1/|S|) Σ_{i∈S} w(y_i) · CrossEntropy(logits_i, y_i)
S = masked positions , w(<fill>) = 0.1 , w(else) = 1
```

`labels = -100` everywhere except `S`, so the same framewise cross-entropy used before now automatically restricts to masked slots. The `<fill>` down-weight (0.1) prevents collapse onto the dominant filler class.

**What happens to each frame:** masked slots are targets (gradient flows); unmasked/given slots are `-100` (ignored as targets but fed as context); the `<fill>` tail and `<pad>` are ignored. "Ignored" means not a prediction target — those frames still shape the attention that fills the masked ones.

## 5.2 Backward

Gradients reach only the trainable parameters — the 8 SA layers, `lm_head`, `mask_embed`, and the text embedding. The frozen HuBERT body receives no gradient. Optimiser and schedule are unchanged from the parent run: AdamW at lr  $2e-4$ , 5000-step warmup, linear decay, weight-decay  $5e-3$ , grad-clip 1.0, bf16 autocast, 4-way DDP, effective batch 256, 100 epochs on LS-960.

## 5.3 Left-alignment and post-`</s>` `<fill>` are supervised

The model does not discover left-packing — every target `y` is left-packed, so it learns "given the whole audio, character `k` goes at position `k`, then `</s>×3`, then `<fill>`." The alignment is **positional, not acoustic**, so the model relies on global bidirectional attention over all audio frames to recover the character order. Because `p` and the per-slot draws are random, every position (chars, `</s>`, and the in-budget `<fill>`) is masked on some steps across the run, so every position's target is learned — including `<mask>` → `<fill>`.

# 6. Inference: OmniVoice iterative unmasking

## 6.1 Canvas length and the scaffold

At inference the audio gives both lengths deterministically: `T = compute_output_length(#samples)` (the full canvas) and `n_keep = ceil(dur · 25)` (the content region). The canvas is seeded:

```
position:  0          1 ... n_keep-1          n_keep ... T-1
token:     <s>      <mask> ... <mask>          <fill> ... <fill>
status:    given   ---- to predict ----      fixed scaffold (given)
```

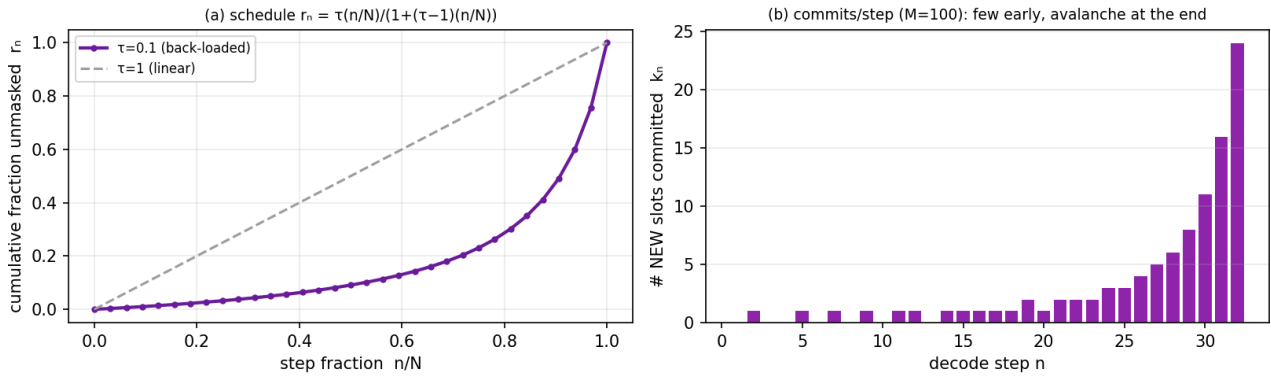
Only the first `n_keep` slots are decoded; the tail is a fixed `<fill>` scaffold that anchors "content ends here." No `<pad>` for a single utterance. This mirrors the training budget exactly, so nothing out-of-distribution is ever predicted.

## 6.2 The schedule

Over `N` steps the number of committed slots follows OmniVoice's time-shifted schedule:

```
r_n = τ · (n/N) / (1 + (τ-1) · (n/N)) ,   r_0=0, r_N=1 ,   k_n = round(r_n · M) - round(r_{n-1} · M)
```

with `τ=0.1` (back-loaded), `M = n_keep - 1`. Few slots commit early — only the highest-confidence anchors and the `<fill>` boundary — and most commit at the end, once context is rich.



### 6.3 One step

Each step re-runs the model on the **whole** current canvas (the frozen tap is cached once):

1. `logp = log_softmax(Model(tap, ids))`.
2. For each still-masked slot: `conf = max_v logp` (confidence), `pred = argmax_v logp` (the value).
3. Optionally subtract a small penalty from `<fill>` confidences so content isn't crowded out.
4. **Sample** `k_n` slots to commit using `conf/temp` as scores (Gumbel-top-k, `temp=5`) — order is stochastic, avoiding a fixed greedy path.
5. **Commit** `ids[slot] = pred` — permanent (monotonic unmask). Value is greedy; only the order is random.

After `N` steps, read out: map `ids` → symbols, cut at the first `</s>`, drop `<fill>/<pad>`, `|` → space.

### 6.4 Commit dynamics

Because `<fill>` slots are trivially high-confidence, the boundary and tail commit early, fixing the utterance length and constraining the content head; letters commit last. This is the opposite of left-to-right — it is confidence-first. Commits are permanent; if early errors hurt WER, a Mask-Predict variant (re-mask the least-confident committed slots) restores self-correction with the same weights.

## 7. Validation and monitoring

Validation runs at two speeds.

**Tier 1 — cheap reconstruction (every eval interval, one forward/clip).** The eval collator uses a deterministic `fixed_p = 1.0`, so `evaluate()` becomes a one-shot ( $N=1$ ) reconstruction: it reports `eval/WER,CER,SER` plus masked-frame accuracy split by class (`content_frame_acc`, `fill_frame_acc`, `fill_pred_rate`, prediction entropy, distinct-non-fill tokens). Low-variance; drives `best.pt`. It is a proxy ( $N=1$ , no `</s>` cap).

**Tier 2 — real iterative decode (periodic / final).** Run the  $N$ -step decoder on dev/test for the true WER/CER/SER. Compare  $N=1$  vs  $N=32$  as a built-in regression check ( $N=1 \approx$  the old model). This is the headline number.

**Series to watch:** `train/loss`, `train/masked_acc`, grad-norm, lr, throughput; the Tier-1 accuracy curves (collapse detector = `fill_pred_rate` ↑ with distinct-non-fill ↓); Tier-2 WER and a periodic WER-vs- $N$  sweep. Eval runs under `model.eval()`, so `mask_embed` and dropout are automatically off — validation reflects true inference conditions.

## 8. How to run

**Train** (4-way DDP, background with logging):

```
ACMLM=1 GPU=6,7,8,9 SA_LAYERS=8 EPOCHS=100 BATCH=32 ACCUM=2 \
  MASK_PROB=0.20 MASK_PROB_MAX=0.30 MASK_LEN=10 EOS_REPEAT=3 FILL_WEIGHT=0.1 \
  OUT=runs/full960_sa8_ACMLM_pe_fps25_mask20-30x10_eos3_fw0.1 MASTER_PORT=29573 \
  bash run.sh
```

**Decode** (iterative, after training):

```
python filler_asr_omni_style_decoding.py \
  --run_dir runs/full960_sa8_ACMLM_pe_fps25_mask20-30x10_eos3_fw0.1 \
  --manifest ../librispeech_test_clean.jsonl --steps 32 --tau 0.1 --temp 5
```

## Appendix — symbol reference

| symbol                         | meaning                                                     |
|--------------------------------|-------------------------------------------------------------|
| T                              | encoder frame count = canvas length (from audio)            |
| n_keep                         | content region length = $\text{ceil}(\text{dur} \cdot 25)$  |
| <s>=30, </s>=31, <fill>=32     | start, end ( $\times 3$ ), left-pack filler                 |
| <pad>=29                       | batch padding (ignored via attention mask)                  |
| <mask>=33                      | input-only placeholder for a hidden slot                    |
| p                              | per-clip masking rate, $\sim U(0, 1)$ (+15% forced 1.0)     |
| $K \sim \text{Binomial}(n, p)$ | number of masked slots per clip                             |
| N, $\tau$ , temp               | decode steps, schedule shift, position-sampling temperature |